



Programa de Residência em Microeletrônica
MCTI / Softex / IRede

ATIVIDADE AVALIATIVA
Controlador de Vending Machine
em SystemVerilog

Trilha: RTL Design — Módulo: Projeto de Controlador Digital

Amanda Vasconcelos e Raissa Genésio

Campina Grande- PB

2026

SUMÁRIO

1 INTRODUÇÃO.....	3
2 OBJETIVOS.....	3
2.1 Objetivo Geral.....	3
2.2 Objetivos Específicos.....	3
3 ESPECIFICAÇÃO DO SISTEMA.....	4
3.1 Funcionamento Geral.....	4
3.2 Entradas e Saídas (Interface Top-Level).....	4
3.3 Regras de Funcionamento.....	4
2 ESPECIFICAÇÃO E ARQUITETURA DO SISTEMA.....	5
2.1 Diagrama de Blocos e Descrição dos Módulos.....	5
3 MÁQUINA DE ESTADOS FINITOS (FSM).....	7
3.1 Diagrama de Transição de Estados.....	7
3.2 Lógica de Controle e Caminho de Dados.....	7
4 VERIFICAÇÃO FUNCIONAL (SIMULAÇÃO).....	9
4.1 Estrutura do Testbench e Resultados do Terminal.....	9
4.2 Resultados: Waveforms Anotadas.....	10
5 SÍNTESE LÓGICA E ANÁLISE DE TIMING.....	11
5.1 Resultados de Área e Frequência Máxima (Tabela de Exploração).....	11
5.2 Análise e Discussão do Caminho Crítico.....	12
6 CONCLUSÃO.....	13

1 INTRODUÇÃO

Este relatório apresenta o projeto, a verificação funcional e a síntese lógica de um controlador síncrono para uma máquina de vendas automática (vending machine). O trabalho foi desenvolvido como atividade prática do módulo de Projeto de Controlador Digital na trilha de RTL Design do Programa de Residência em Microeletrônica (CiExpert), com o objetivo de consolidar conceitos de integração entre unidade de controle e caminho de dados (datapath).

O fluxo adotado segue os padrões da indústria de semicondutores: a modelagem RTL foi descrita em SystemVerilog, a validação funcional conduzida via simulação automatizada com o Synopsys VCS (com suporte do Verdi para análise de formas de onda) e a viabilidade física avaliada via síntese lógica com o Synopsys Design Compiler, utilizando restrições temporais (SDC) para determinar a área e a frequência máxima do circuito.

Todo o código-fonte, scripts e arquivos de restrições estão disponíveis em:

Link	para	o	Repositório	GitHub:
https://github.com/RaissaGenesio/Grupo_AmandaRaissa_vending.git				

2 OBJETIVOS

2.1 Objetivo Geral

Desenvolver uma máquina de vendas automática (Vending Machine), baseada em uma Máquina de Estados Finitos (FSM) do tipo Moore, capaz de realizar o controle de crédito, seleção de produtos, verificação de estoque, liberação do item adquirido e cálculo do troco, além de validar seu funcionamento por meio de simulação e síntese digital.

2.2 Objetivos Específicos

- Implementar a arquitetura em SystemVerilog, utilizando FSM Moore na unidade de controle.
- Desenvolver os módulos de armazenamento de crédito, memória de produtos, comparação de crédito e cálculo do troco.
- Integrar todos os módulos em um componente top-level.
- Criar um testbench para validação funcional do sistema e verificar o funcionamento via simulação.
- Realizar a síntese lógica do projeto e analisar os resultados obtidos.

3 ESPECIFICAÇÃO DO SISTEMA

3.1 Funcionamento Geral

O controlador da *vending machine* é estruturado sob o conceito de separação clara entre **Unidade de Controle** e **Caminho de Dados** (*datapath*). O comportamento sequencial e as decisões lógicas são coordenados por uma Máquina de Estados Finitos (FSM) síncrona do tipo Moore. Os demais blocos atuam no processamento aritmético e de armazenamento, operando na acumulação de créditos, consulta síncrona a uma memória interna de produtos e cálculo de subtração do troco.

3.2 Entradas e Saídas (Interface Top-Level)

O sistema recebe como entradas o sinal de clock (*clk*), utilizado para sincronizar todas as operações, e o sinal de reset (*rst*), responsável por inicializar a máquina em seu estado de repouso. A entrada *coin_in* informa o valor da moeda inserida pelo usuário, enquanto *sel_item* identifica o produto desejado. Os sinais *confirm* e *cancel* permitem, respectivamente, confirmar a compra ou cancelar a operação.

As saídas do sistema indicam o resultado da operação realizada. O sinal *dispense* informa quando o produto deve ser liberado, *change_out* apresenta o valor do troco calculado, *display* representa o crédito acumulado pelo usuário, *error* sinaliza condições de erro e *state_out* disponibiliza o estado atual da máquina para fins de monitoramento e depuração.

3.3 Regras de Funcionamento

1. Após o reset, a máquina fica em espera, aguardando a inserção de moedas.
2. O crédito é atualizado a cada moeda válida inserida, sendo possível acumular várias antes da compra.
3. Ao selecionar um produto e confirmar, o sistema verifica preço e estoque.
4. A venda só é autorizada se houver estoque e crédito suficiente.
5. Autorizada a venda, o produto é liberado, o estoque atualizado e o troco calculado (crédito – preço).
6. Se o usuário cancelar antes da conclusão, o crédito é devolvido e a máquina volta ao estado inicial.
7. Com crédito insuficiente ou produto indisponível, o sistema entra em estado de erro até nova operação.

2 ESPECIFICAÇÃO E ARQUITETURA DO SISTEMA

2.1 Diagrama de Blocos e Descrição dos Módulos

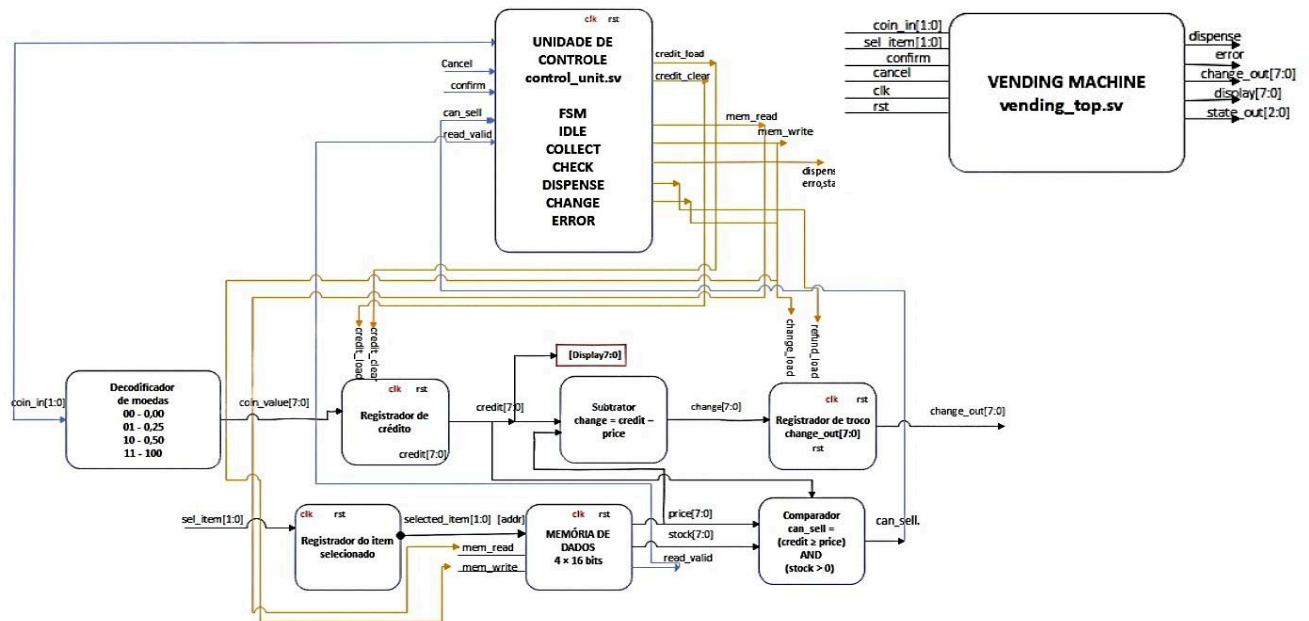


Figura 01: Diagrama de Blocos da Vending Machine

1. Módulo principal — vending_top.sv

O módulo *vending_top.sv* é o top-level do sistema, responsável por reunir todos os blocos e interligar seus sinais. Representa a interface completa entre o usuário e o circuito, mostrando as entradas e saídas do projeto. Ele instancia o registrador de crédito, a memória, o comparador, o subtrator e a unidade de controle. Foi colocado como bloco externo porque encapsula toda a comunicação do sistema com o ambiente.

2. Package de definições — vending_pkg.sv

O arquivo *vending_pkg.sv* centraliza constantes e definições globais, como os estados da FSM. Não aparece como bloco funcional no diagrama, pois serve como arquivo de apoio para padronizar nomes e codificações, evitando o uso de números soltos no código.

3. Decodificador de moedas

Converte a entrada *coin_in[1:0]* em um valor de 8 bits (*coin_value[7:0]*). A entrada chega codificada em 2 bits, mas o restante do sistema opera com valores em centavos usando 8 bits. O decodificador faz a ponte entre o código da moeda e o valor real a ser somado. Foi incluído

para que o registrador de crédito não precise interpretar os códigos 01, 10 ou 11, recebendo diretamente um valor numérico.

4. **Registrador de crédito — credit_reg.sv**

Registrador síncrono de 8 bits que acumula o crédito do usuário. Quando a FSM ativa credit_load, soma-se coin_value ao crédito atual; com credit_clear (reset, cancelamento ou fim da compra) o valor é zerado. Foi colocado no sistema porque o crédito é uma informação central, e sem esse registrador a máquina não conseguiria acumular várias moedas antes da confirmação da compra.

5. **Registrador do item selecionado**

Armazena a escolha do usuário (sel_item[1:0]) no momento da confirmação, gerando o sinal selected_item usado como endereço da memória. Foi incluído para garantir que o item escolhido permaneça estável durante as operações de compra, impedindo que mudanças acidentais na entrada corrompam a consulta e aumentando a robustez do circuito.

6. **Memória de dados — memory.sv**

Memória com 4 posições de 16 bits: 8 bits superiores para o preço e 8 bits inferiores para o estoque dos produtos. A FSM controla quando ler ou escrever, mas os dados ficam separados da lógica de controle. Foi colocada porque a máquina precisa consultar preço e estoque antes de liberar um produto, e essa separação deixa o projeto mais organizado.

7. **Comparador — comparator.sv**

Gera o sinal can_sell avaliando continuamente se o crédito é suficiente ($\text{credit} \geq \text{price}$) e se há estoque ($\text{stock} > 0$). Representa a lógica combinacional de decisão de venda. Foi colocado no diagrama para simplificar o trabalho da FSM, que no estado CHECK apenas recebe o resultado pronto, sem precisar fazer cálculos. Implementá-lo isoladamente é adequado por não exigir armazenamento.

8. **Subtrator de troco — subtractor.sv**

Calcula de forma contínua a diferença $\text{change} = \text{credit} - \text{price}$, obtendo o valor do troco. Faz parte do caminho de dados essencial da máquina após uma compra válida. A escolha por um bloco combinacional se justifica porque o cálculo não precisa de armazenamento interno; o resultado flui continuamente e só é salvo quando a FSM ativa a carga no estado CHANGE.

9. **Registrador de troco**

Armazena o valor que será apresentado em change_out[7:0], podendo ser o troco calculado ou

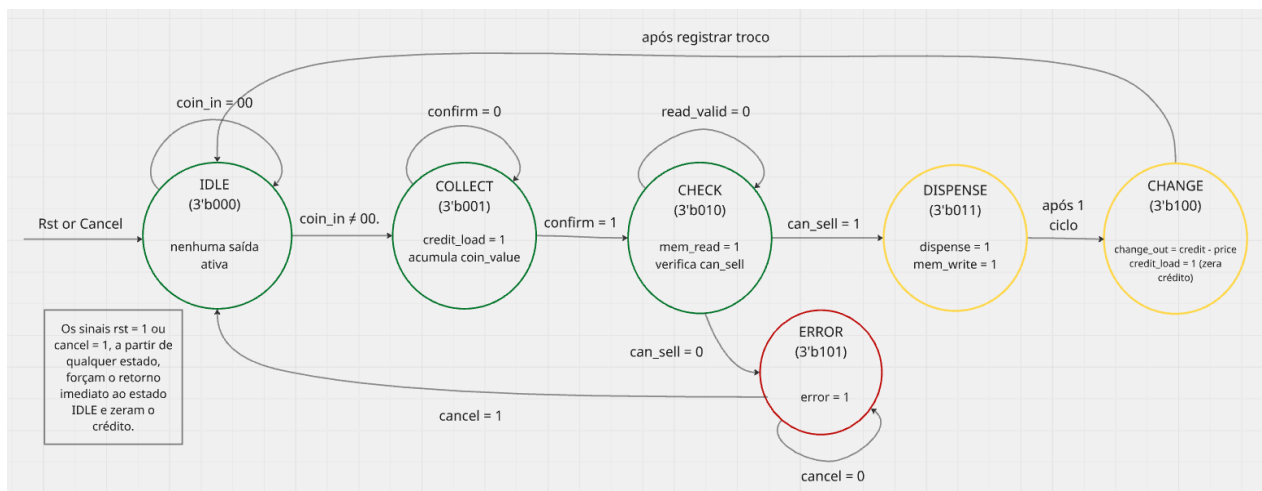
o crédito integral em caso de cancelamento. Foi incluído porque a saída precisa manter um valor válido mesmo depois que o crédito interno for zerado, evitando mudanças indesejadas. Essa separação entre cálculo e apresentação final registra a devolução no momento exato e mantém o resultado estável para o usuário.

10. Unidade de controle — `control_unit.sv`

Implementa a máquina de estados finitos (FSM) que coordena as etapas de coleta, verificação, liberação e troco. É o “cérebro” do sistema: os outros blocos armazenam ou calculam dados, enquanto a FSM interpreta sinais e gera todos os comandos de controle, definindo quando cada operação deve ocorrer. Essa escolha é exigida pelo funcionamento sequencial da máquina, que precisa orquestrar o caminho de dados passo a passo.

3 MÁQUINA DE ESTADOS FINITOS (FSM)

3.1 Diagrama de Transição de Estados



3.2 Lógica de Controle e Caminho de Dados

A arquitetura do controlador é baseada na intensa troca de sinais entre a FSM e o Caminho de Dados. A seguir, descreve-se o fluxo de controle estado a estado:

- **IDLE (3'b000):** Estado de repouso. A FSM não ativa nenhuma saída e aguarda passivamente a inserção de moedas. A transição ocorre quando detecta `coin_in != 2'b00`.

- **COLLECT (3'b001):** Estado de acúmulo. A cada ciclo em que uma moeda válida é inserida ($\text{coin_in} \neq 2'b00$), a FSM ativa o sinal de controle $\text{credit_load} = 1$. Isso comanda o Registrador de Crédito a somar o valor da moeda ao saldo atual. O estado é mantido até que o usuário ative $\text{confirm} = 1$.
- **CHECK (3'b010):** Estado de verificação, com duração exata de um ciclo de clock. A FSM ativa o sinal $\text{mem_read} = 1$ para consultar o preço e o estoque do produto na memória de dados. Simultaneamente, o Comparador (no Datapath) avalia essas variáveis matemáticas e retorna a flag combinacional can_sell . Com base nisso flag, a FSM decide seu próximo passo: avança para DISPENSE se $\text{can_sell} = 1$, ou desvia para ERROR se $\text{can_sell} = 0$.
- **DISPENSE (3'b011):** Estado de liberação do produto. A FSM gera o pulso combinacional de um ciclo $\text{dispense} = 1$ para o hardware físico externo. Ao mesmo tempo, aciona $\text{mem_write} = 1$ para ordenar à Memória de Dados que decremente o estoque do item vendido. A transição para o próximo estado ocorre incondicionalmente no ciclo seguinte.
- **CHANGE (3'b100):** Estado de finalização da compra. A FSM comanda o Subtrator a atualizar o troco, ativando $\text{change_out} = \text{credit} - \text{price}$. Imediatamente, ativa $\text{credit_load} = 1$ para forçar o crédito a zero e preparar a máquina para um novo cliente, retornando automaticamente ao estado IDLE.
- **ERROR (3'b101):** Estado de exceção, alcançado quando há falta de estoque ou de crédito. A máquina trava sua operação e ativa $\text{error} = 1$ para alertar o usuário. A FSM permanece neste estado indefinidamente, até que o usuário pressione $\text{cancel} = 1$, o que devolve o dinheiro e restaura o estado IDLE.

4 VERIFICAÇÃO FUNCIONAL (SIMULAÇÃO)

4.1 Estrutura do Testbench e Resultados do Terminal

```
=====
CENARIO 1 - COMPRA COM TROCO
=====
PASS: FSM chegou ao estado DISPENSE | estado=011
PASS: dispense ativo | esperado=1 obtido=1
PASS: dispense dura somente um ciclo | esperado=0 obtido=0
PASS: FSM retornou ao estado IDLE | estado=000
PASS: troco da compra do cafe | esperado=75 obtido=75
PASS: credito zerado apos a compra | esperado=0 obtido=0

=====
CENARIO 2 - CREDITO INSUFICIENTE
=====
PASS: FSM chegou ao estado ERROR | estado=101
PASS: sinal error ativo | esperado=1 obtido=1
PASS: cancelamento retirou FSM do ERROR | estado=000
PASS: credito insuficiente devolvido | esperado=25 obtido=25
PASS: credito zerado apos cancelamento | esperado=0 obtido=0
```

```
=====
CENARIO 3 - CANCELAMENTO DE R$2,00
=====
PASS: credito acumulado antes do cancelamento | esperado=200 obtido=200
PASS: FSM voltou para IDLE apos cancelamento | estado=000
PASS: devolucao de R$2,00 | esperado=200 obtido=200
PASS: credito zerado pelo cancelamento | esperado=0 obtido=0

=====
CENARIO 4 - ESGOTAMENTO DO ESTOQUE DO CAFE
=====
PASS: compra de cafe antes do esgotamento | estado=011
PASS: dispense nas compras com estoque | esperado=1 obtido=1
PASS: retorno ao IDLE apos compra de cafe | estado=000
PASS: compra de cafe antes do esgotamento | estado=011
PASS: dispense nas compras com estoque | esperado=1 obtido=1
PASS: retorno ao IDLE apos compra de cafe | estado=000
PASS: compra de cafe antes do esgotamento | estado=011
PASS: dispense nas compras com estoque | esperado=1 obtido=1
PASS: retorno ao IDLE apos compra de cafe | estado=000
PASS: compra de cafe antes do esgotamento | estado=011
PASS: dispense nas compras com estoque | esperado=1 obtido=1
PASS: retorno ao IDLE apos compra de cafe | estado=000
PASS: sexta tentativa chegou ao ERROR | estado=101
PASS: erro por estoque zerado | esperado=1 obtido=1
PASS: produto nao liberado sem estoque | esperado=0 obtido=0
PASS: FSM final retornou ao IDLE | estado=000
```

```
=====
RESUMO DA SIMULACAO
PASS = 31
FAIL = 0
=====
RESULTADO FINAL: TODOS OS TESTES PASSARAM
$finish called from file "../tb_vending.sv", line 279.
$finish at simulation time 670000
V C S   S i m u l a t i o n   R e p o r t
Time: 670000 ps
CPU Time: 0.420 seconds; Data structure size: 0.0Mb
Thu Jul 9 21:50:47 2026
```

Observa-se que todos os cenários executados foram aprovados, totalizando 31 verificações bem-sucedidas e nenhuma falha. Esse resultado demonstra que o projeto atende aos requisitos funcionais especificados para a vending machine, incluindo compra com troco, tratamento de crédito insuficiente, cancelamento da operação e detecção de estoque esgotado.

4.2 Resultados: Waveforms Anotadas

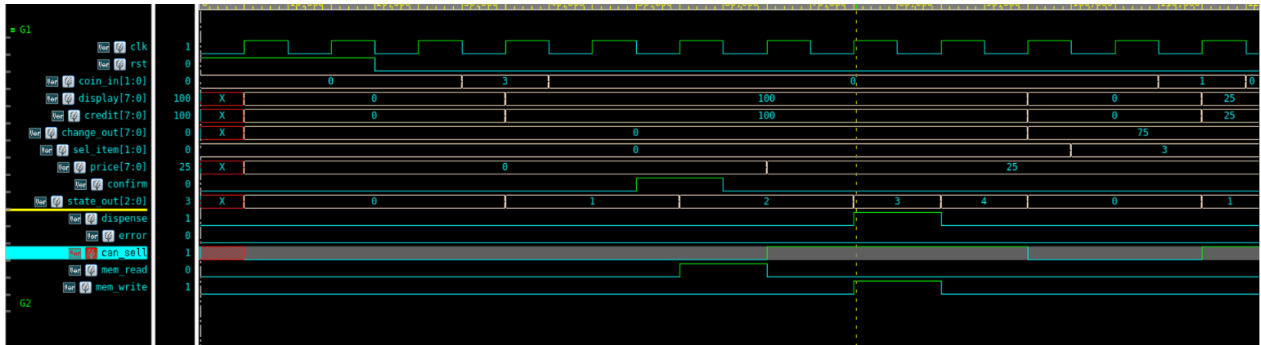


Figura 02: Cenário 1 (Compra com Troco)

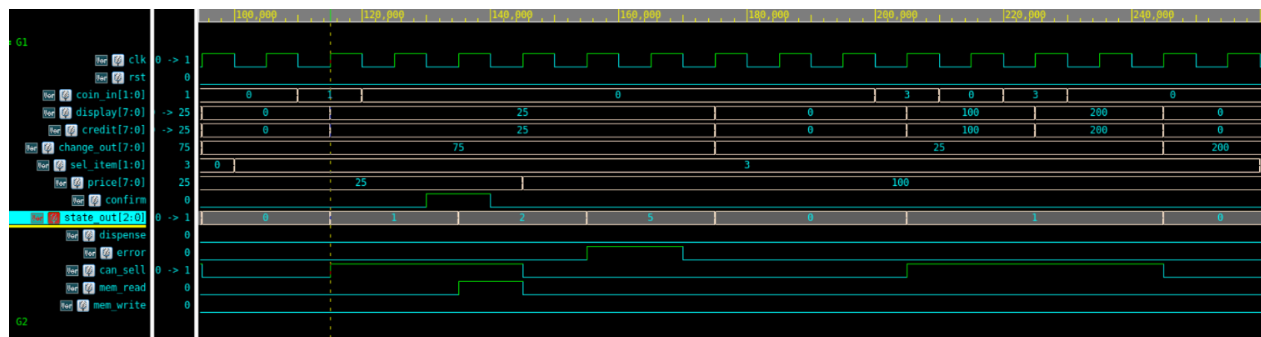


Figura 03: Cenário 2 (Crédito Insuficiente)

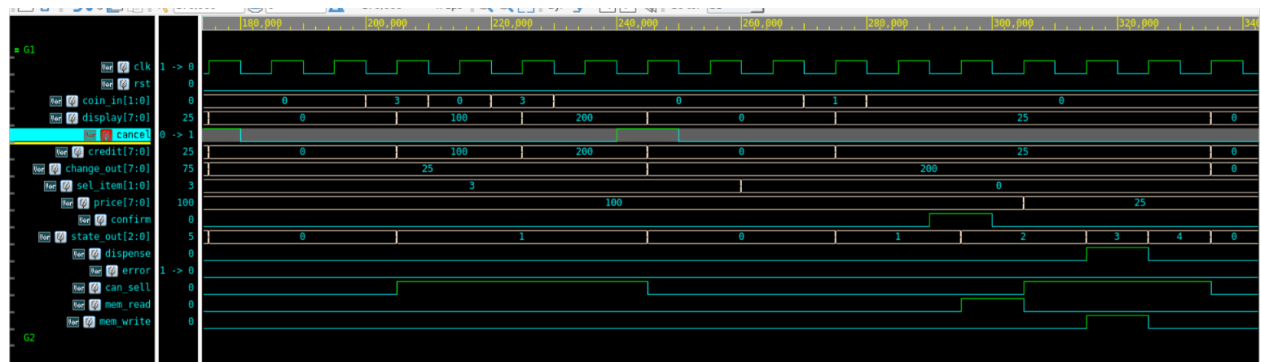


Figura 04: Cenário 3 (Cancelamento)

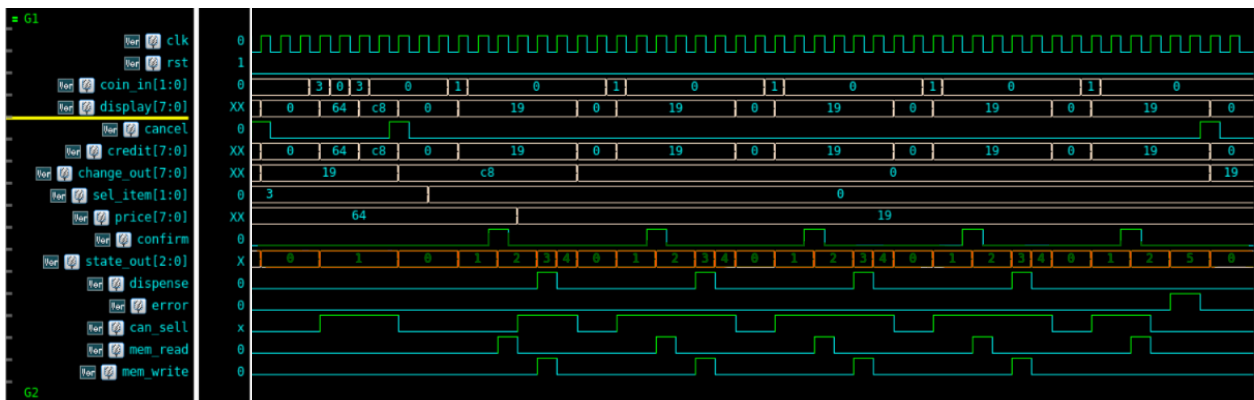


Figura 05: Cenário 4 (Estoque Zerado)

5 SÍNTESE LÓGICA E ANÁLISE DE TIMING

5.1 Resultados de Área e Frequência Máxima (Tabela de Exploração)

Período	Frequência	Slack(ns)	Área	Timing
20 ns	50 MHz	16.01 ns	1017.83	MET
18 ns	55.6 MHz	14.01 ns	1017.83	MET
16 ns	62.5 MHz	12.01 ns	1017.83	MET
14 ns	71.4 MHz	10.01 ns	1017.83	MET
12 ns	83.3 MHz	08.01 ns	1017.83	MET
10 ns	100 MHz	06.01 ns	1017.83	MET
8 ns	125 MHz	04.01 ns	1017.83	MET
6 ns	166.7 MHz	-0,69	1017.83	VIOLATED

Foi realizada uma exploração da frequência máxima de operação do circuito por meio da redução progressiva do período do clock. O projeto atendeu todas as restrições temporais até um período de **8 ns**, correspondente a uma frequência de **125 MHz**. Ao reduzir o período para **6 ns (166,67 MHz)**, foram observadas violações de setup, indicando que essa frequência excedeu o limite operacional do circuito sintetizado.

5.2 Análise e Discussão do Caminho Crítico

1. Qual é o caminho crítico do design? Quais módulos ele atravessa?

Pela análise do relatório de timing gerado pelo Design Compiler, observou-se que o pior caminho termina em um registrador do módulo vending_memory (mem_reg[3][0]), passando pela lógica do control_unit, responsável pela geração do sinal mem_write.

Durante a análise temporal, percebeu que o caminho crítico atravessou os seguintes módulos:

- Control_Unit - onde são gerados os sinais de controle da máquina de estados;
- vending_memory, responsável pela atualização do estoque dos produtos

Esse caminho é o que determina a escrita na memória de estoque após a confirmação da compra, sendo o trecho de maior atraso do projeto.

2. Como a área total variou à medida que o período foi reduzido? Explique o motivo.

A área do circuito permaneceu praticamente constante. O relatório final apresentou:

- Área das células: 859,52 μm^2
- Área total (incluindo interconexões): 1017,83 μm^2

Mesmo reduzindo o período do clock de 20 ns até 8 ns, o Design Compiler conseguiu atender às restrições temporais sem necessidade de redimensionar células ou realizar otimizações adicionais que aumentassem a área do circuito.

Isso ocorreu porque o projeto possuía uma margem temporal elevada. O pior caminho apresentava aproximadamente 3,46 ns de atraso, enquanto o período mínimo que ainda atendia às restrições era significativamente maior. Sendo assim, o sintetizador não precisou substituir portas por versões maiores ou inserir buffers adicionais.

3. A partir de qual frequência o Design Compiler não conseguiu mais fechar o timing? O que acontece com o slack nesse ponto?

A partir do timing 6 ns (166,67 MHz) surgiram violações temporais nas saídas dispense e error, indicando que o circuito não conseguiu atender ao tempo requerido. O que acontece é que o slack se torna negativo, indicando que os dados chegam após o instante exigido pelo clock, impossibilitando o funcionamento correto nessa frequência.

4. Considerando a FSM e o caminho de dados do projeto, qual estado ou transição você acredita estar no caminho crítico

Considerando as análises, provavelmente o estado de transição que é responsável pela confirmação da compra. Provavelmente porque é a sequência que envolve simultaneamente a lógica da máquina de estados e o caminho de dados da memória, concentrando a maior quantidade de lógica combinacional em um único ciclo de clock.

5. Que modificação no RTL você proporia para reduzir o caminho crítico sem alterar o comportamento funcional do design?

Uma alternativa seria realizar pipeline na operação de compra.

No projeto atual, diversas operações são executadas no mesmo ciclo de clock, incluindo a verificação do crédito, a atualização do estoque e a geração dos sinais de saída. Essas operações poderiam ser divididas em dois ciclos distintos:

1. No primeiro ciclo seriam realizadas as verificações de crédito e disponibilidade do produto;
2. No segundo ciclo ocorreria a atualização da memória e a geração dos sinais de dispensação.

Essa ideia seria viável porque reduziria a quantidade de lógica combinacional presente em um único ciclo, conseqüentemente diminuindo o atraso máximo do circuito e permitindo operar em frequências mais elevadas.

6 CONCLUSÃO

O desenvolvimento deste projeto permitiu aplicar, na prática, os conceitos estudados. Durante sua realização, foi possível passar por todas as etapas do desenvolvimento de um circuito digital, desde a definição da arquitetura e implementação em SystemVerilog até a verificação funcional. Os resultados obtidos demonstraram que o circuito implementado atende aos requisitos propostos, apresentando funcionamento correto durante as simulações e atendendo às restrições de temporização após a etapa de síntese.

Ao longo do desenvolvimento, algumas dificuldades foram encontradas. A divisão das tarefas e a comunicação entre os integrantes da equipe exigiram organização para que os módulos

desenvolvidos individualmente pudessem ser integrados corretamente. Outro desafio foi o aprendizado das ferramentas utilizadas, principalmente o ambiente Synopsys, que possui diversos comandos, arquivos de configuração e relatórios que precisaram ser compreendidos ao longo da atividade. Também houve dificuldades relacionadas à depuração de erros, à interpretação dos relatórios de síntese e ao gerenciamento do tempo para conciliar implementação, testes, correções e documentação do projeto.

Apesar desses desafios, a atividade proporcionou diversos aprendizados. Foi possível compreender melhor o funcionamento de máquinas de estados, da modelagem de circuitos em SystemVerilog e da organização de projetos em módulos hierárquicos. Além disso, a construção do testbench e a análise dos resultados da síntese permitiram relacionar os conceitos vistos em sala de aula com uma aplicação prática. De modo geral, o projeto contribuiu para o desenvolvimento de conhecimentos técnicos e também de habilidades importantes, como organização, trabalho em equipe, comunicação e resolução de problemas.